Documentação Detalhada da API: iScrev XBoard

Bem-vindo à documentação oficial da API Interna do **iScrev XBoard**. Este documento foi redigido de forma didática para ajudar novos desenvolvedores a entenderem como as engrenagens da lousa digital se conectam.

O XBoard funciona através de um conjunto de **Classes Singleton** (instanciadas uma única vez e anexadas ao objeto global `window`), que interagem de maneira assíncrona.

---

1. Engine Gráfica: `window.Engine` (`CanvasEngine`)

O `CanvasEngine` é o coração visual da lousa. Ele captura eventos de caneta/mouse e desenha linhas ultra-suaves no `<canvas>` usando Curvas Quadráticas de Bézier.

Acessando as Propriedades Internas:

- `Engine.canvas` / `Engine.ctx` : O elemento DOM e o contexto 2D com `desynchronized: true`.
- `Engine.currentTool` : Retorna `'pen'` (caneta) ou `'eraser'` (borracha).
- `Engine.currentColor` e `Engine.currentLineWidth` : Estado atual do traço.

Métodos Principais:

- **`setTool(tool)`** : Alterna a ferramenta. *Exemplo:* *`window.Engine.setTool('eraser');`*
- **`setColor(hex)` / `setLineWidth(px)`** : Atualiza as configurações da caneta.
- **`redrawHistory(strokes)`** : Fundamental para desfazer/refazer e trocar de páginas. Esse método limpa completamente o Canvas e desenha do zero todo o array de traços passados via argumento.
- **`clear()`** : Limpa fisicamente o `<canvas>`. (Nota: Não apaga a memória, apenas a tela. Para apagar os dados definitivamente, use o `HistoryManager`).

---

2. Máquina de Estado: `window.HistoryManager` (`History`)

Se o Engine é o coração gráfico, o `HistoryManager` é o cérebro lógico. Ele gerencia as múltiplas páginas ("Pranchetas" ou "Boards") da sua lousa digital.

Estrutura de uma Prancheta (Board):

Cada prancheta da lousa tem o seguinte formato:

```
{
  "id": 1690000000000,
  "strokes": [ /* Array com todos os tracados */ ],
  "redoStack": [ /* Tracados desfeitos esperando Redo */ ],
  "bg": { "type": "solid", "color": "#1e1e1e" }
}
```

Métodos de Desenho e Edição:

- **`addStroke(stroke)`** : Chamado automaticamente pelo `CanvasEngine` ao soltar o mouse. Anexa o traço atual na prancheta, zera o `redoStack` e ativa o save automático (debounce).
- **`undo()` / `redo()`** : Retira o último traço do `strokes` e joga para o `redoStack` (ou vice-versa), chamando imediatamente o `window.Engine.redrawHistory()` para re-renderizar a tela.
- **`clearAll()`** : Remove todos os `strokes` da prancheta e esvazia a tela.

Métodos de Paginação:

- `addBoard()` : Cria uma folha em branco baseada na cor de fundo da folha atual e pula para ela.
- `removeBoard()` : Remove a prancheta atual e volta para a folha anterior (se houver).
- `nextBoard() / prevBoard()` : Navega entre as folhas da aula. Se o usuário der "Próximo" na última folha, uma nova folha em branco é automaticamente gerada.

---

3. Banco de Dados: `window.StorageService` (`StorageService`)

Faz a ponte entre a memória RAM (`HistoryManager`) e o HD do usuário usando **IndexedDB**, garantindo que a lousa funcione off-line e sobreviva a um encerramento inesperado.

API de Sessão (Lousa Ativa):

O XBoard mantem a sessão ativa salva em "tempo real" (1 segundo de intervalo).

- `saveSession(data)` : Sobrescreve o ID fixo `'current'` com os dados de todas as páginas da lousa atual. Invocado em *background* pelo `HistoryManager` .
- `loadSession()` : Roda automaticamente ao iniciar o App, restaurando sua lousa de onde parou.

API de Gerenciamento de Aulas:

- `saveLessonAs(title, callback)` : "Fotografa" todas as páginas da lousa atual e salva como um projeto com título e data no IndexedDB. Útil para professores que criam templates.
- `getAllLessons(callback)` : Retorna um array com todas as aulas salvas, ordenadas das mais recentes para as mais antigas.
- `loadLesson(id, callback)` : Carrega o projeto passado no argumento diretamente para a tela.
- `deleteLesson(id, callback)` : Exclui definitivamente um projeto da memória do navegador.

---

4. Gravador Nativo: `window.RecorderModule` (`Recorder`)

A API de estúdio embutida no aplicativo. Utiliza as tecnologias `MediaRecorder` e `captureStream` (a 30 FPS).

Como a Gravação Funciona:

Como o XBoard tem um `main-canvas` (traços transparentes) e um `bg-canvas` (fundo com grade/cores), o `Recorder` aciona um loop de animação e cria um **terceiro Canvas Oculto (MergeCanvas)** que mescla os 2 quadros em tempo real na hora de gravar.

- Ao mesmo tempo, ele chama `navigator.mediaDevices.getUserMedia` requisitando acesso ao **microfone** do instrutor.
- Se o usuário rodar no Electron, o microfone é aprovado automaticamente pelo sistema operacional.

Métodos:

- `toggleRecording()` : Liga/desliga o gravador. Se estiver desligado, roda internamente o `startRecording()` .
- `stopRecording()` : Interrompe o loop de mesclagem visual e consolida o arquivo.
- `exportVideo()` : Chamada logo após o `stop` . Gera um *Blob* `.webm` a partir dos "chunks" gravados e força o download da aula no computador do professor.

---

5. Integração na Interface: `window.UIManager`

Não contém lógica de negócio pesada, mas age como o **maestro dos eventos visuais**.

- Todos os `document.getElementById('btn-...')` são registrados aqui.
- Invoca métodos do `Engine` quando clica numa cor, ou o `HistoryManager` quando o usuário pressiona "Avançar Prancheta".
- Se você criar um novo botão HTML amanhã, o seu `addEventListener` deverá ser cadastrado dentro do método `bindEvents()` do `UIManager`.

> ***Dica para Refatoração:*** *Ao programar novos módulos, evite acessar o DOM diretamente de dentro do* `History` *ou* `Engine`*. Sempre delegue chamadas de Interface para os métodos expostos pelo* `UIManager` *(ex:* `window.UIManager.updateBoardCounter(cur, max)`*).*

---

6. Tabela Resumo Geral da API

Módulo (Instância Global)	Responsabilidade Principal	Métodos / Propriedades Chave
`CanvasEngine` (`window.Engine`)	Motor gráfico e renderização vetorial (Bézier).	`setTool()`, `setColor()`, `redrawHistory()`, `clear()`
`History` (`window.HistoryManager`)	Máquina de estados, paginação e controle Undo/Redo.	`addStroke()`, `undo()`, `redo()`, `nextBoard()`, `changeBoard()`
`StorageService` (`window.StorageService`)	Persistência local no IndexedDB (sessões e projetos).	`saveSession()`, `saveLessonAs()`, `loadLesson()`, `getAllLessons()`
`Recorder` (`window.RecorderModule`)	Estúdio de gravação nativo (merge de canvas + áudio).	`toggleRecording()`, `startRecording()`, `stopRecording()`, `exportVideo()`
`UIManager` (`window.UIManager`)	Maestro de interface, capturando eventos e atualizando a tela.	`updateBoardCounter()`, `updateBgColor()`, delegação de clicks

---

7. Mapeamento de Módulos e Objetos por Arquivo (Detalhado)

Esta tabela estrutura a relação entre os arquivos físicos da base de código, as Classes ES6 contidas neles e descreve de forma minuciosa os **métodos de interface e seus parâmetros de entrada**.

Arquivo Fonte	Classe / Instância	Objeto Associado	Métodos e Parâmetros (Assinatura

`src/js/core/canvas-engine.js`	`CanvasEngine` (`window.Engine`)	Renderização (`Strokes`, `Canvas`)	`setTool(tool)`
			`setColor(color)`
			`setLineWidth(width)`
			`setBgColor(color)`
			`onPointerDown(e)`, `onPointerMove(e)`, o
			`applyContextStyles(pressure)`
			`clear()`
			`redrawHistory(strokes)`

`src/js/core/history.js`	`History` (`window.HistoryManager`)	Estados e `Boards`	`addStroke(stroke)` `undo()`, `redo()`

			clearAll() setBg(config) addBoard(), removeBoard() nextBoard(), prevBoard() changeBoard() redraw() saveToStorage() loadFromStorage(data)
src/js/services/storage.js	StorageService (window.StorageService)	Sessões, IndexedDB	initDB() saveSession(sessionData) loadSession() saveLessonAs(title, callback) getAllLessons(callback) loadLesson(id, callback) deleteLesson(id, callback)
src/js/modules/recorder.js	Recorder (window.RecorderModule)	Fluxos Áudio/Vídeo, MediaRecorder	toggleRecording() startRecording() renderLoop() stopRecording() exportVideo()
src/js/modules/ui-manager.js	UIManager (window.UIManager)	Interface e DOM	bindEvents() updateBoardCounter(current, total) updateBgColor(config)
src/js/modules/background-manager.js	BackgroundManager (window.BackgroundManager)	Texturas e Fundo	render(ctx, width, height, bgConfig drawLines(ctx, width, height) drawGrid(ctx, width, height)
src/js/modules/i18n.js	I18n (window.I18n)	Dicionário	setLang(lang) t(key)

			translateDOM()
src/js/modules/pdf-exporter.js	PdfExporter (window.PdfExporter)	Instância jsPDF	exportLesson(id) generatePdf(boards, title) *(assíncro*
main.js & preload.js	Processos Main e Renderer do Node	IPC (electronAPI)	createWindow() session.defaultSession.setPermissio setDisplayMediaRequestHandler(reque